RHEL 9 환경에서의 패키지 및 사용자 관리 실습

DNF · RPM · Repository · 사용자/그룹 · 권한 관리

RHEL 9 / VMware Fusion 실습

1. 패키지 관리의 큰 그림
2. DNF와 RPM 역할 구분
3. 저장소와 실습 명령어
4. 사용자·그룹·권한 실습

오늘의 학습 지도

개념 → 명령어 → 해석 → 실습 → 점검의 흐름으로 이해합니다

01

패키지 관리의 큰 그림

패키지·저장소·DNF·RPM의 관계

→

02

DNF vs RPM

설치/업데이트는 DNF, 조회/검증은 RPM

→

03

Repository 이해

BaseOS, AppStream, repo 파일 구조

→

04

사용자/그룹

계정 생성, 그룹 부여, 변경, 삭제

→

05

권한과 소유권

chmod, chown, chgrp와 결과 해석

핵심 포인트: root로 모든 것을 처리하기보다 일반 사용자 + sudo 관점까지 함께 봐야 실습의 의미가 살아납니다.

패키지 관리의 큰 그림

패키지는 프로그램 묶음이고, Repository는 그 패키지를 보관하는 창고입니다

패키지 (package)

설치 가능한 프로그램 묶음
예: bash, tree, httpd

→

Repository (repo)

패키지와 메타데이터를 모아 둔 저장소

→

DNF

repo를 보고 패키지를 검색·설치·업데이트·삭제

→

RPM

설치 여부·파일 목록·패키지 정보 조회와 검증

- 실제 설치 동작은 DNF가 주도하고, 설치 기록과 세부 정보는 RPM 계층이 관리합니다.
- 초보자는 “DNF로 설치하고, RPM으로 확인한다”는 규칙부터 잡으면 이해가 쉽습니다.
- Repository 메타데이터는 패키지 이름, 버전, 의존성 같은 정보를 담고 있습니다.

DNF와 RPM, 무엇이 다른가?

둘 다 패키지 관리에 관여하지만 쓰는 순간과 목적이 다릅니다

핵심 비교

구분	DNF	RPM
수준	고수준 관리자	저수준 도구
주요 역할	검색·설치·업데이트·삭제	질의·검증·파일 목록 확인
의존성	자동 해결	직접 해결 필요할 수 있음
대표 명령	dnf install, dnf update	rpm -q, rpm -ql, rpm -qf

```
sudo dnf install tree
rpm -q tree
rpm -ql tree
```

실습 해석

- 설치는 DNF가 담당: repo에서 tree를 찾아 의존성까지 같이 처리
- 확인은 RPM이 담당: 설치 여부와 설치된 파일 목록을 질의
- 즉, “동작은 DNF / 추적은 RPM”이라는 관점으로 기억하기

시험/실습 팁: 설치는 DNF, 확인은 RPM

Repository 구조 이해

RHEL 9에서는 BaseOS와 AppStream 저장소를 중심으로 패키지를 찾고 설치합니다

핵심 개념

- Repository : 패키지 파일과 메타데이터를 함께 보관하는 저장소
- BaseOS : 운영체제 핵심 패키지
- AppStream : 응용 소프트웨어와 런타임을 주로 제공
- 활성화된 저장소는 dnf repolist, repo 설정 파일은 /etc/yum.repos.d/에서 확인합니다.

```
dnf repolist
ls /etc/yum.repos.d/
cat /etc/yum.repos.d/redhat.repo
```

패키지 설치·업데이트·삭제 실습 흐름

실습에서는 가벼운 패키지인 tree를 예시로 쓰면 이해가 빠릅니다

1 검색

```
dnf search tree
```

→

2 정보 확인

```
dnf info tree
```

→

3 설치

```
sudo dnf install -y tree
```

→

4 확인

```
tree
rpm -ql
```

→

5 삭제

```
sudo dnf remove -y tree
```

- 설치·업데이트·삭제는 repo를 보는 DNF가 담당합니다.
- 설치 결과가 제대로 반영됐는지 확인할 때는 RPM 질의 명령이 유용합니다.
- check-update는 “업데이트 후보”, update는 “실제 반영”으로 구분해서 생각하세요.

명령어 출력, 이렇게 읽으면 됩니다

초보자가 막히는 지점은 “무슨 뜻인지 모르겠다”는 순간입니다

예시 1: dnf repolist

```
$ dnf repolist
repo id                                  repo name
rhel-9-for-aarch64-baseos-rpms           Red Hat Enterprise Linux 9 BaseOS
rhel-9-for-aarch64-appstream-rpms        Red Hat Enterprise Linux 9 AppStream
```

- repo id: 내부 식별자
- repo name: 사람이 읽는 이름
- BaseOS / AppStream이 보이면 기본 repo 인식 성공

예시 2: rpm -ql tree

```
$ rpm -ql tree
/usr/bin/tree
/usr/share/doc/tree
/usr/share/man/man1/tree.1.gz
```

- 실행 파일이 어디에 설치됐는지 확인 가능
- 문서와 매뉴얼 파일까지 패키지 소속으로 추적 가능
- “이 파일이 어느 패키지 것이지?”는 rpm -qf로 확인

사용자와 그룹: 왜 나눠서 관리할까?

사용자는 사람 또는 서비스의 정체성, 그룹은 권한을 묶어서 배포하는 단위입니다

- useradd: 사용자 계정 생성
- groupadd: 그룹 생성
- usermod -aG: 사용자를 보조 그룹에 추가
- id / groups: 사용자의 UID, GID, 소속 그룹 확인
- 일반 사용자 + sudo 관점으로 실습해야 권한 차이가 보입니다.

```
sudo useradd alice
sudo groupadd devteam
sudo usermod -aG devteam alice
id alice
```

사용자·그룹 실습: 생성 → 변경 → 삭제

실습은 작은 계정을 만들고, 그룹에 넣고, 확인하고, 정리하는 순서가 좋습니다

생성

```
sudo useradd alice
sudo groupadd devteam
sudo passwd alice
```

사용자와 그룹을 만든 뒤 비밀번호를 설정합니다.

변경

```
sudo usermod -aG devteam alice
id alice
getent group devteam
```

보조 그룹 추가 후 id / getent로 반영 여부를 확인합니다.

삭제

```
sudo userdel -r alice
sudo groupdel devteam
# 홈 디렉터리까지 삭제
```

userdel -r는 홈 디렉터리까지 제거하므로 주의합니다.

확인 포인트: /etc/passwd, /etc/group, id, groups, getent

초보자 실수: root로만 작업해서 일반 사용자 권한 차이를 놓치는 것

파일 권한과 소유권, 어떻게 읽을까?

권한 문자열은 소유자(user) · 그룹(group) · 기타(other)의 순서로 읽습니다

```
-rw-r-----   1 alice  devteam
120  file1
```

파일 종류 | 소유자 권한 | 그룹 권한 | 기타 권한

기호	의미	숫자값
r	읽기 (read)	4
w	쓰기 (write)	2
x	실행 (execute)	1
-	권한 없음	0

예: 640 = rw- r-- ---

- chmod: 권한 자체를 바꿈
- chown: 소유자(또는 소유자:그룹)를 바꿈
- chgrp: 그룹만 바꿈
- 디렉터리는 읽기보다 "진입/탐색 권한 x"의 의미를 함께 봐야 합니다.

chmod · chown · chgrp 실습

같은 파일을 놓고 "권한 변경"과 "소유권 변경"을 구분해 보는 것이 핵심입니다

chmod

```
chmod 640 file1
chmod u+x file1
chmod g-w file1
```

권한 비트만 바꿉니다. 숫자 방식과 기호 방식 둘 다 익혀야 합니다.

chown

```
sudo chown alice file1
sudo chown alice:devteam file1
```

소유자 또는 소유자:그룹을 함께 바꿉니다. root 권한이 필요한 경우가 많습니다.

chgrp

```
sudo chgrp devteam file1
```

그룹만 바꾸고 싶을 때 사용합니다. chown과 목적이 다릅니다.

함께 확인: ls -l file1 / ls -ld dir1 / id alice

실수 포인트: 파일 권한만 바꿨는데 소유권까지 바뀌었다고 착각하지 않기

초보자가 자주 헷갈리는 포인트

실습 실패는 대부분 “개념 구분이 흐려진 순간”에 발생합니다

DNF와 RPM을 같은 도구로 생각	→ DNF는 설치/업데이트/삭제, RPM은 조회/검증 중심으로 구분
Repository와 패키지를 혼동	→ repo는 창고, package는 창고 안의 물건이라는 비유로 기억
root로만 모든 실습 수행	→ 일반 사용자 + sudo 관점이 있어야 권한 차이가 보임
chmod와 chown을 혼동	→ chmod는 권한 비트, chown은 소유자/그룹
삭제 명령의 영향 범위 간과	→ userdel -r는 홈 디렉터리까지 삭제하므로 신중히 사용
출력 결과를 안 읽고 넘어감	→ id, ls -l, dnf repolist, rpm -q 결과를 꼭 해석해 보기

발표 포인트: “왜 그런지”를 함께 설명하면 암기 부담이 크게 줄어듭니다

전체 요약

이번 시간의 다섯 문장만 기억해도 실습 흐름이 잡힙니다

- 패키지는 설치 가능한 프로그램 묶음이고, Repository는 그 패키지를 모아 둔 저장소입니다.
- RHEL 9에서는 DNF가 기본 패키지 관리 도구이며, 설치·업데이트·삭제를 담당합니다.
- RPM은 설치 여부, 파일 목록, 특정 파일의 소속 패키지를 확인할 때 특히 유용합니다.
- 사용자와 그룹은 권한을 효율적으로 배포하기 위한 기본 구조입니다.
- chmod, chown, chgrp는 각각 “권한”, “소유자”, “그룹”을 바꾸는 명령입니다.

한 줄 규칙

설치는 DNF
확인은 RPM
권한 실습은 일반 사용자 중심

실습 체크리스트

실습 전·중·후에 아래 항목을 확인하면 대부분의 문제를 스스로 해결할 수 있습니다

실습 시작 전

- ☐ subscription-manager identity 정상 확인
- ☐ dnf repolist로 활성 repo 확인
- ☐ sudo 권한이 있는 일반 사용자 준비
- ☐ 실습 전 VMware 스냅샷 생성
- ☐ tree 같은 가벼운 패키지로 먼저 연습

실습 중 / 종료 후

- ☐ id / groups / getent로 사용자·그룹 확인
- ☐ ls -l / ls -ld로 권한과 소유권 해석
- ☐ rpm -q / rpm -ql / rpm -qf를 함께 사용
- ☐ userdel -r, dnf remove는 영향 범위 확인
- ☐ 마지막에 실습 계정과 파일 정리